Artificial Intelligence Practicum

Emergency Response System

Planning emergency responses after a storm

Lorek Jeremi
Student number: 20231362

GitHub repository

https://github.com/JeremiLrk/AI-Emergency-Response-System

November 2026

1. System overview and probabilistic reasoning

The aim of this project was to decide which emergency responders to send after a storm, and how they should reach each incident. Finding a short route is only one part of that problem. A responder needs the right skills, may already be busy, and might need a road that is blocked. Calls also appear during the simulation, so an assignment that looks sensible now can leave the wrong responder in the wrong place later.

I based the design on the supplied scenario files and developed the components separately before connecting them. The files distinguish two kinds of uncertainty: Bayesian road conditions are Open, Congested and Blocked, while the HMM describes Low, Medium and High traffic. The traffic readings are historical, with no simulation timestamps. I kept these distinctions explicit so the models would not accidentally describe the same thing or use information at the wrong time.

Estimating road conditions

The Bayesian model connects weather, road condition and noisy reports: Weather → Road condition → Reports. As covered in the Bayesian networks chapter, I combine the network's probabilities and normalize them using the known evidence:

$$P(Q \mid E = e) = \frac{P(Q, E = e)}{P(E = e)}.$$

Here Q is the road condition we want to estimate, and $E = e$ represents the weather and reports we know. For each condition, I multiply its weather-based probability by how likely the available reports would be under that condition. Dividing the three weights by their total gives the final probabilities. Missing reports add no evidence. If the weather is unknown, I consider each possible weather condition and weight it by its given probability. Each road direction is estimated separately.

For example, on N1 → N2 in Scenario 1, the weather is Rain. Citizen and Police report Blocked, while Sensor reports Congested. The weights for Open, Congested and Blocked are $(0.00012, 0.00384, 0.00441)$. Dividing by their sum, 0.00837, gives approximately $(0.0143, 0.4588, 0.5269)$. Blocked is the most likely state, but treating it as certain would throw away the remaining uncertainty. I therefore kept the full distribution and made road use a separate planning decision.

By default, the system avoids roads with a 20% or higher chance of being blocked. This limit can be changed, since a lower limit leaves fewer roads available. For roads it accepts, the system estimates travel time assuming they are not blocked. An Open road takes the normal travel time: distance divided by speed. A Congested road takes 1.5 times the normal travel time. The estimate combines these two times, giving more weight to the condition that is more likely.

This way of estimating travel time is a design choice for this project. It assumes the road is not blocked, so the predicted arrival time is not guaranteed. The Bayesian model also treats each report as separate evidence. If one source copies another, the model may count the same information twice and become too certain about the road condition.

Learning from historical traffic

For each road, I use an HMM to estimate the probabilities of Low, Medium and High traffic from its historical readings. Since a reading can be wrong, the model keeps all three probabilities instead of treating one traffic level as certain.

For example, H2 → N8 in Scenario 2 has the readings Low, Medium, High, Medium and High. I start with the scenario's initial probabilities: 50% Low, 35% Medium and 15% High. The first reading is Low. Using the emission table, the model checks how likely that reading would be under each traffic level and updates the probabilities. Dividing the results by their total makes them add up to one.

For the next reading, Medium, the model first uses the transition table to predict how traffic could have changed. It then uses the emission table to update that prediction with the reading. These are the two Forward updates from the HMM chapter:

$$B'(X_{t+1}) = \sum_{x_t} P(X_{t+1} \mid x_t) B(x_t), \qquad B(X_{t+1}) \propto P(e_{t+1} \mid X_{t+1}) B'(X_{t+1}).$$

Here, B holds the current traffic probabilities and B' holds the prediction before including the next reading. The symbol $\propto$ means the updated values still need to be divided by their total.

Repeating these steps for all five readings gives about 1.0% Low, 28.2% Medium and 70.8% High. Routing can then use this road's probability of High traffic as part of its traffic preference.

A road without readings keeps its scenario's initial probabilities. I process all histories before the simulation starts and keep the results fixed during the run. The readings have an order, but no simulation timestamps, so each Forward step represents another historical reading rather than another simulated second.

2. Routing and responder allocation

The road estimates give a directed graph with nonnegative travel costs. Uniform Cost Search (UCS) explores the cheapest path so far, ordering its queue by accumulated cost $g(n)$. Cheaper discoveries update earlier paths. The destination is accepted when removed from the queue, since first discovering it need not give the cheapest route. UCS is optimal for these estimates, while real conditions remain unknown.

I also tested UCS against A*, another search method from the course. A* uses $f(n) = g(n) + h(n)$: the travel time so far plus an estimate of the time still needed. In this project, that estimate is the straight-line distance to the

destination divided by the highest road speed. The code checks that the road data supports this estimate, since it must never be higher than the shortest remaining travel time. Both algorithms found the same minimum travel times in the tests. UCS is still used for normal runs; I did not compare their speed.

Using traffic estimates in routing

I use the Bayesian model to estimate travel time and the HMM to help choose between routes. Applying another slowdown based on the HMM could count congestion twice. It could also make an Open road slower than the normal travel time specified in the assignment.

Instead, I give each route a traffic score. For each road, I multiply its normal travel time by its probability of High traffic, then add the results for the whole route. A lower score means less travel along roads likely to have heavy traffic. This score is a design choice for this project. It does not represent extra travel time or the chance of a blockage.

Using this score only when two routes take exactly the same time would rarely affect the choice. I therefore also allow a slightly slower route when it has a clearly lower traffic score: up to 5% more estimated travel time for at least a 10% lower score.

In strict mode, the selected route must still reach the incident before or at its deadline. Its estimated travel time determines arrival and when the responder becomes available again. The traffic score is only used to help choose the route.

Choosing between competing calls

The simulator builds possible responder–incident pairs from the current situation. Only free responders with the required skill and a usable route qualify. OR-Tools CP-SAT then chooses a compatible set, using Boolean variables and sum constraints as seen in the CSP chapter, adapted to responder assignments. For each eligible pair, $x_{ri} = 1$ means send responder r to incident i, and 0 means do not send them. Our assignment constraints are

$$\sum_i x_{ri} \leq 1 \quad \text{for each responder,} \qquad \sum_r x_{ri} \leq 1 \quad \text{for each incident.}$$

These rules ensure that each responder receives at most one new assignment at a time and each incident gets at most one responder. I first maximize the number of Critical incidents assigned, then High, Medium and Low incidents, in that order. Only after that does the solver minimize response time. Otherwise, assigning nobody would give a total response time of zero and could appear to be the best solution. I do not give extra preference to an earlier deadline when both incidents can still be reached on time. The solver handles these goals one at a time and only locks in a result when it has proved that result is the best possible for that goal. If it runs out of time, it keeps a valid assignment already found, or returns no assignments if none was found. It does not claim that an unfinished search found the best solution. Since CP-SAT needs integer values, response times are rounded up to milliseconds. The simulation still uses the original travel times, including fractional seconds.

Simulation and availability

The normal run only allows assignments expected to arrive on time. An optional allow-late mode can still send help after a deadline. In that mode, the solver first maximizes assignments by priority, then on-time arrivals by priority, and finally minimizes response time. A call can therefore be handled but still count as late.

The simulation jumps to the next event, such as a new call, an arrival or the end of treatment. Events at the same time are handled together, including jobs that take zero time. The allocator only sees calls already released and responders currently available. A responder stays busy through travel and treatment, then becomes free at the incident location. Once dispatched, an assignment cannot be changed.

run.py saves the assignments in plan.json and the results in statistics.txt. A separate validator uses the original scenario data to check routes, skills, timing and deadlines without relying on the simulator's calculations.

3. Evaluation and reflection

All 175 tests pass. I tested each component separately and also ran the complete system on all three scenarios. Bayesian probabilities are checked against calculations done by hand. For the HMM and small allocation problems, tests try every possible state sequence or assignment and compare the answers with Forward and CP-SAT. UCS and A* return the same shortest travel times for 1,690 pairs of scenario locations. Both also match the shortest paths found by trying every possible simple path in 750 small-graph cases. Other tests check missing or incorrect data, one-way roads, unreachable incidents, busy responders and deadline rules.

I compared CP-SAT with a simpler method that assigns responders based on the shortest travel time, without considering incident priority. Both methods use the same road, traffic and deadline settings. The table below shows the results with strict deadlines and a 20% road blockage cutoff.

Scenario	CSP handled	Nearest handled	CSP on time	CSP Mean response (s)
1	6/7	5/7	85.7%	91.1
2	11/12	11/12	91.7%	33.3
3	13/20	15/20	65.0%	607.9

An incident counts as handled once treatment is finished. Average response time measures how long handled calls waited from release until a responder arrived, including waiting before dispatch. The on-time percentage includes

every incident, so unhandled calls count as missed deadlines. All times are in seconds. With late responses allowed, CP-SAT handled all 7, 12 and 20 incidents, but missed 2, 2 and 9 deadlines. Helping every call therefore did not mean reaching every call on time.

In Scenario 3, CP-SAT handled 13 of 20 incidents, while the nearest-responder method handled 15. An early choice between responders with equal travel times left them at different locations later. Because those responders had different skills, this affected which later calls they could reach on time. CP-SAT optimizes the current assignments, but does not plan several jobs ahead. The simpler method happened to produce better coverage in this scenario.

When testing routes between all locations, the HMM changed the suggested route in seven cases in Scenario 3. None of these journeys was needed during the simulation, so the HMM did not change the actual assignments or results.

My role in the LLM collaboration

I used ChatGPT for design discussions and Codex for implementation and testing. I steered the project by checking suggestions against the assignment, questioning assumptions and deciding which changes to make. The LLM helped work out the details and turn those decisions into code. The exact first substantive prompt is included after the report.

ChatGPT's early explanation mixed Bayesian road conditions with HMM traffic levels and treated historical observations as if they arrived during the simulation. I pointed out that the scenario files describe two different things and that the traffic readings are already available before the run. The design was corrected to keep these models separate and process the history once.

ChatGPT proposed choosing the most likely road condition. I preferred keeping all three probabilities because the most likely condition is not certain. I also kept the blockage cutoff adjustable, since a strict limit could remove too many routes.

For traffic integration, ChatGPT suggested multiplying Bayesian and HMM slowdown factors or taking the larger factor. I challenged this because it could count congestion twice and make Open roads slower than the assignment specifies. I chose a separate traffic score instead. I also questioned using that score only when route times were exactly equal, which led to the limited-detour option.

During the allocation design, I raised the problem that minimizing response time alone could reward assigning nobody. I chose to maximize assignments by priority first. ChatGPT also proposed ranking earlier deadlines before response time. I rejected that extra stage because an earlier deadline alone should not outweigh a shorter response when both calls can be reached on time.

I challenged the strict-deadline design because a late response can still be useful. The final system therefore keeps strict mode as the default but also supports late responses, with deadline results recorded separately. I also required the simulator to hide future calls and correctly handle initially busy responders and simultaneous events.

I wanted clearer explanations of unhandled calls, but required them to reuse existing checks without changing planning. I specifically distinguished calls with no suitable candidate from calls that lost a responder to another assignment. Comparing 48 runs confirmed unchanged results. I also replaced confusing command-line use with a scenario menu and requested console results and Critical coverage counts.

Testing, I also caught generated code using Responder.location instead of start_location. This was corrected.

I suggested allowing roads with a higher chance of blockage for Critical calls when the normal routes cannot reach them on time. ChatGPT first considered this separately for each call. I pointed out that two Critical calls might need the same responder, so the decision should consider both calls together. Another responder might reach one of them using a higher-risk route, allowing both calls to be handled. I also questioned a rule that would prevent helping an extra Critical call if it meant handling fewer lower-priority calls. Finally, I noted that checking only the riskiest road does not tell us the risk of the whole route. I did not implement this idea.

What I would improve next

The simulator currently assumes that responders can follow their planned routes. It does not model what happens if they encounter a blocked road. I would extend it so responders can discover a blockage and find another route. This would show whether the plans still work when road conditions are different from the estimates.

I would also test the proposed higher-risk routes for Critical calls. The comparison should show both how many extra calls are reached on time and how much additional risk is accepted. A higher number of handled calls alone would not be enough to judge whether this is a better policy.

Another question is what should happen after treatment. Responders currently become available at the incident location. In practice, an ambulance may first need to take a patient to a hospital, while other responders may return to their station. I would model these extra journeys and the time they take before a responder becomes available again. I would also compare returning to a station with staying near the last incident, since their location affects how quickly they can reach the next call.

Original first substantive LLM prompt

The following prompt is reproduced without rewriting. This additional prompt page is separate from the three content pages.

> I've attached my AI practicum assignment, the scenarios and the course slides.
>
> From what I understand, we need to build a system that assigns emergency responders to incidents and finds suitable routes, while taking into account deadlines, responder availability and uncertain road conditions.
>
> We need to combine search, CSPs, Bayesian reasoning and HMMs to do this. Before starting the implementation, I want to figure out how these components should work together and how to structure the project.
>
> Since the assignment is open-ended, I'd like to discuss the possible approaches and important design decisions first. Use the course slides as the main reference, but feel free to suggest alternatives and point out problems with my approach.
>
> The goal is to build a working system that I can fully understand, test and defend afterwards.